



Практика 8. Полиморфизм подтипов (II)

Реализуйте в C++ полиморфную иерархию контейнеров, например, как аналог иерархии коллекций в C#.

Каждый контейнер реализовывать как обёртку над соответствующим контейнером STL.

По максимуму используйте исключения.

Ниже приведён упрощённый и адаптированный вариант требуемой иерархии из стандартной библиотеки C#.

Итератор `IEnumerator<T>`:

```
interface IEnumerator<T> {  
    public bool MoveNext();  
    public T Current();  
}
```

Интерфейс итерабельного объекта `IEnumerable<T>`:

```
interface IEnumerable<T> {  
    public IEnumerator<T> GetEnumerator();  
}
```

Интерфейс коллекции `ICollection<T>`:

```
interface ICollection<T> : IEnumerable<T> {  
    public int Count();  
    public void Add(T item);  
    public void Clear();  
    public bool Contains(T item);  
    public bool Remove(T item);  
}
```

Динамический массив `List<T>`:

```
class List<T> : ICollection<T> {  
    // ... (всё, что требуется, согласно реализуемым интерфейсам)  
  
    public int Capacity();  
    public void SetCapacity(int capacity);  
  
    // operator[]  
    public T this[int index] { get; set; }  
  
    public void Insert(int index, T item);  
    public void RemoveAt(int index);  
}
```

Неупорядоченное множество `HashSet<T>`:

```
class HashSet<T> : ICollection<T> {  
    // ...  
  
    public int Capacity();  
    public void SetCapacity(int capacity);  
}
```

Для элементов хеш-таблицы необходимо задать хеш-функцию и компаратор.

В С# по дизайну метод `GetHashCode` есть у любого объекта, а компаратор `IEqualityComparer<T>` может быть задан с помощью соответствующей [перегрузки конструктора](#).

При реализации на С++ выберите один из двух вариантов:

1. (Подход С++) Используйте дополнительные шаблонные параметры `Hash` и `KeyEqual` у класса хеш-таблицы, как в `std::unordered_set`.
2. (Подход С#) Предоставьте соответствующую перегрузку конструктора с параметрами, позволяющими задать хеш-функцию и компаратор, и сохраните их в поля класса как `std::function`.

Неупорядоченный словарь `Dictionary<TKey, TValue>`:

```
class Dictionary<TKey, TValue> : ICollection<KeyValuePair<TKey, TValue>> {  
    // ...  
  
    public int Capacity();  
}
```

```
public void SetCapacity(int capacity);

// operator[]
public TValue this[TKey key] { get; set; }
```

Стек `Stack<T>`:

```
class Stack<T> : IEnumerable<T> {
    // ...

    public int Count();

    public void Push(T item);
    public T Peek();
    public T Pop();
}
```

Очередь `Queue<T>`:

```
class Queue<T> : IEnumerable<T> {
    // ...

    public int Count();

    public T Dequeue();
    public T Peek();
    public void Enqueue(T item);
}
```